HARSH JHA

+91 7002714761 Bangalore , India harshjha8453@gmail.com GitHub LinkedIn Leetcode

SUMMARY

Backend-focused developer skilled in Java and Spring Boot, with strong understanding of REST APIs, concurrency, and distributed transaction patterns such as Saga and idempotency.

EDUCATION

Master of Computer Applications (MCA) Jain University
Expected June 2026

Bachelor of Computer Applications (BCA) Girijananda Chowdhury University
2020–2023

SKILLS

Languages	Java, SQL
Backend	Spring Boot, REST APIs
Concepts	Idempotency, Saga Pattern, Concurrency, Async Processing
Databases	MySQL (JPA/Hibernate)
DevOps	AWS (EC2, S3 - basic), Docker, CI/CD (GitHub Actions)
Problem Solving	Practicing DSA (Arrays, Hashing, Basic Patterns)
Tools	Git, Postman

EXPERIENCE

Backend Developer Intern Aug 2025 – Dec 2025
Invicto Labs, Bangalore

- Developed backend services using Node.js, Express.js, and MySQL for core features and integrations
- Designed and implemented REST APIs to improve integration efficiency across systems
- Refactored codebase into modular architecture, enhancing maintainability and scalability
- Automated data extraction workflows using Puppeteer, reducing manual effort

Backend-focused Full-Stack Developer Intern Feb 2025 – Jul 2025
Triot Solution, Remote

- Built backend services using Java Spring Boot and PostgreSQL for a production-ready system
- Implemented JWT-based authentication for secure and stateless API access
- Applied layered architecture to improve code structure and extensibility
- Developed basic responsive UI using React.js and Tailwind CSS to support backend features

PROJECTS

Nivora Pay — Distributed Wallet System *Spring Boot, MySQL, ShardingSphere*
GitHub Link

- Built idempotent wallet transfers ensuring exactly-once execution and eliminating duplicate debit scenarios
- Designed Saga-based workflow to guarantee consistency across multi-step transactions (debit, credit, status update)
- Ensured concurrency safety using database-level locking to prevent race conditions in balance updates
- Implemented asynchronous processing to reduce API latency and enable non-blocking transaction execution
- Scaled data handling using ShardingSphere-based database sharding for distributed load management